

RESEARCH DEVELOPMENT PLAN

Knowledge-Grounded LLM Agent for Causal Discovery

Using Reactome Ontology and the Sachs Protein Signalling Dataset

Mikołaj Jarosławski, Dominik Sepiolo, Antoni Ligęza

AGH University of Krakow — Department of Applied Computer Science

May 2026

1. Scientific Background and Motivation

1.1 Research Lineage

This project sits at the convergence of three research lines developed at AGH University of Krakow, each of which provides a direct foundation for the proposed system.

Model-Driven XAI (MD-XAI). Sepiolo and Ligeza (Applied Sciences, 2024) established the 3C principle - Causality, Components, Connections - as the foundation for white-box, model-driven explainability. The framework argues that shallow post hoc methods such as LIME and SHAP are fundamentally limited because they operate on the same input data as the ML model and cannot incorporate external declarative knowledge. The proposed alternative uses Grammatical Evolution guided by domain knowledge to discover analytical, interpretable models. Causal structure discovery is identified as a prerequisite step for any meaningful MD-XAI pipeline.

Causal Discovery Benchmarking. Jaroslowski, Sepiolo, and Ligeza (RAI 2025) performed a systematic empirical evaluation of PC, GES, and LiNGAM on the LUCAS dataset using the causal-learn library. The study demonstrated that domain knowledge constraints (mandatory and forbidden edges) improve LiNGAM performance meaningfully (AUPR: 0.312 to 0.495; Recall: 0.33 to 0.58), while PC and GES showed limited sensitivity. The study established the experimental protocol -- metrics (AUPR, SHD, Precision, Recall, F1), algorithm stack, and ablation design -- that will be directly inherited by this project.

LLM-Augmented Evolutionary Computation. Sepiolo, Jaroslowski, and Ligeza (RAI 2025) demonstrated that LLMs can iteratively refine grammars for Grammatical Evolution, reducing the need for expert intervention in symbolic regression tasks. The paper shows both the potential and the current limitations of LLM-in-the-loop scientific workflows, concluding that partial human intervention was still required to obtain correct solutions.

1.2 Related Work: LLM-Augmented Causal Discovery

A growing body of literature investigates the integration of LLMs with classical causal discovery algorithms. The most closely related systems are surveyed below.

System / Paper	Knowledge Source	Key Mechanism	Gap vs. This Work
CausalCopilot (Wang et al., 2025)	Implicit (LLM parametric)	End-to-end automation: algorithm selection, hyperparameter tuning, post-processing with LLM pruning and direction revision	Knowledge is implicit and non-auditable; no structured ontology; no variable grounding
ALCM (Khatibi et al., 2024)	LLM-extracted priors	LLM provides domain knowledge constraints before statistical causal discovery runs	No structured knowledge base; single-source; no grounding module
CATE-B (Liu et al., 2024)	Literature queries (post hoc)	LLM queries scientific literature to resolve edge ambiguities after initial discovery	Literature queried only post hoc for disambiguation; no pre-discovery ontology

System / Paper	Knowledge Source	Key Mechanism	Gap vs. This Work
This Work	Structured ontology (Reactome) + LLM reasoning	Automatic variable grounding + Cypher-based ontology query + LLM causal reasoning from reaction roles → graded constraints	Tri-source knowledge, grounded and auditable; generalises across datasets in the domain

1.3 The Core Novelty Argument

Existing systems that use LLMs for causal discovery rely on implicit, parametric knowledge encoded in the LLM's weights during pretraining. This knowledge is unverifiable, domain-unspecific, and cannot be updated or audited. The proposed system instead uses a curated, peer-reviewed biological ontology (Reactome) as its primary knowledge source. The LLM acts as a reasoning bridge: it maps dataset-specific variable names to ontology concepts (variable grounding) and then interprets structured reaction participation data (inputs, outputs, catalysts, regulators) to infer directional causal claims with confidence scores.

This approach enables three claims that no existing system can make simultaneously: (1) causal constraints are derived from an auditable, domain-authoritative source; (2) the system generalises to any proteomic dataset in human biology without manual re-engineering; and (3) the quality of ontology-derived constraints can be empirically measured against a known ground-truth causal graph.

1.4 Dataset: Sachs Protein Signalling

The Sachs et al. (2005) dataset records simultaneous measurements of 11 phosphorylated proteins and phospholipids in human immune cells under both observational and interventional conditions. It has 7,466 instances, 11 nodes, and 17 directed arcs in its ground-truth DAG, established through targeted molecular interventions. It is the canonical real-world benchmark for causal discovery, directly used as an example in DoWhy, CDT, and dodiscover. The biological domain (MAPK signalling, calcium signalling, PI3K/Akt pathway) is well covered by Reactome, making the ontology query layer both feasible and information-rich.

Loading via CDT: `from cdt.data import load_dataset; data, graph = load_dataset('sachs')`

2. System Architecture Overview

The system is a sequential agentic pipeline with four logical stages. Each stage is implemented as a distinct Python module so that components can be tested and ablated independently.

#	Stage	Input → Output	Key Technology
1	Variable Grounding	Column names + dataset description → {column: UniProt_ID} mapping	LLM structured output (JSON mode)
2	Ontology Query	UniProt ID pairs → reaction subgraphs with typed roles (input, output, catalyst, regulator)	Reactome Neo4j (Docker) + Cypher queries via neo4j Python driver
3	Causal Reasoning	Reaction context text → (cause, effect, confidence, reasoning_trace) per variable pair	LLM with chain-of-thought; structured JSON output
4	Constrained Discovery	Graded constraints + observational data → discovered DAG + evaluation metrics	causal-learn (PC, GES, LiNGAM) + BackgroundKnowledge API

3. Development Steps

Each step below is self-contained, produces a testable artifact, and maps to a distinct module in the codebase. Steps are ordered by dependency: each step assumes the previous is complete and tested.

STEP 1	Environment Setup and Sachs Baseline	Foundation
Goal	Reproduce the PC / GES / LiNGAM baseline on Sachs; establish evaluation harness	
Output	baseline_results.json with SHD, AUPR, Precision, Recall, F1 for all three algorithms	
Depends on	Nothing — this is the starting point	

Background

This step directly extends the experimental protocol from Jarosławski et al. (RAI 2025), which established the same three algorithms on the LUCAS dataset. Reproducing comparable baselines on Sachs ensures that improvements from knowledge injection are measured on a consistent foundation. The Sachs dataset is loaded via the Causal Discovery Toolbox (CDT), which provides both the observational data ($7,466 \times 11$ continuous measurements) and the ground-truth NetworkX DiGraph.

Implementation Tasks

- Install dependencies: causal-learn, cdt, neo4j Python driver, anthropic, networkx
- Load Sachs via CDT: data, graph = load_dataset('sachs')
- Implement evaluation harness: compute SHD, AUPR, Precision, Recall, F1 given predicted and true graphs
- Run PC, GES, LiNGAM with default hyperparameters; repeat 10 times with fixed seeds
- Store results in structured JSON; verify they are plausible (PC SHD should be ~17 without priors)

Agent Prompt — Baseline Evaluation Harness

SYSTEM
You are a scientific ML engineer implementing a causal discovery research pipeline in Python. Your code must be correct, modular, and reproducible. Use causal-learn 0.1.3+ and CDT. Always include docstrings and type hints. Output must be deterministic given fixed random seeds.
USER
Implement a Python module `baseline.py` that does the following: 1. Loads the Sachs protein signalling dataset using CDT:<pre>from cdt.data import load_dataset data, true_graph = load_dataset('sachs')</pre> 2. Runs three causal discovery algorithms from causal-learn on the data: - PC (alpha=0.05, ci_test='fisherz')

```

- GES (score_func='local_score_BIC')
- LiNGAM (DirectLiNGAM)

3. Evaluates each result against the ground-truth graph using these metrics:
   SHD, AUPR, Precision, Recall, F1-Score

4. Repeats each algorithm 10 times with seeds 0..9, stores mean and std for each
   metric.

5. Saves all results to baseline_results.json in the following schema:
   { 'algorithm': {'SHD': {'mean': float, 'std': float}, ... } }

The evaluation function must accept two networkx.DiGraph objects (predicted, true)
and return a dict of metric names to float values.
Handle the conversion from causal-learn's GeneralGraph to networkx DiGraph cleanly.
Print a formatted summary table to stdout when the module is run directly.

```

STEP 2	Reactome Neo4j Setup and Schema Exploration	Foundation
Goal	Deploy Reactome locally via Docker; understand the schema needed for protein-pair queries	
Output	Working Neo4j connection; validated Cypher query for retrieving reaction context between two proteins	
Depends on	Step 1 (requires knowing Sachs variable names)	

Background

Reactome is a free, peer-reviewed, open-data knowledgebase of human biological pathways. Its Neo4j graph database encodes reactions as nodes with typed outgoing relationships: `[:input]`, `[:output]`, `[:catalystActivity]`, and `[:regulatedBy]`. The graph hierarchy is: `Pathway` → `ReactionLikeEvent` → `PhysicalEntity` → `ReferenceEntity` (which carries UniProt identifiers). This structure means that two proteins can be linked through their co-participation in reactions, with the type of participation (input, output, catalyst, regulator) encoding the mechanistic direction of their relationship — the raw material from which the LLM will reason about causal direction.

Reactome provides an official Docker image hosted on AWS ECR, which is the recommended and most reproducible deployment method. The image contains Neo4j 4.x and a pre-loaded Reactome database, accessible at `localhost:7474` (browser) and `localhost:7687` (Bolt protocol, used by the Python driver).

Implementation Tasks

- Pull and run the Reactome Docker image; verify Neo4j browser is accessible
- Install the neo4j Python driver: `pip install neo4j`
- Explore schema: list all node labels and relationship types in the database
- Manually identify the UniProt IDs for the 11 Sachs proteins (RAF1, MAP2K1, ERK1/2, PKA, PKC, PIP2, PIP3, Akt, PLCg, P38, JNK)
- Write and validate a Cypher query that retrieves all reactions where two given UniProt IDs co-participate, returning their roles

- Test the query for 3 known Sachs edge pairs (e.g., PKC/RAF1, RAF1/MEK1) and verify biologically plausible results are returned

Agent Prompt — Reactome Connection and Query Module

SYSTEM

You are a scientific ML engineer building a Python interface to a local Reactome Neo4j database.
 Reactome is running in Docker at bolt://localhost:7687 with default credentials (neo4j/reactome).
 The graph schema: Pathway -[:hasEvent]-> ReactionLikeEvent -
 [:input|:output|:catalystActivity|:regulatedBy]-> PhysicalEntity
 PhysicalEntity -[:referenceEntity]-> ReferenceEntity (has
 .identifier = UniProt ID, .geneName list)
 Write correct, efficient Cypher. Always filter for human reactions using [:species {taxId: '9606'}].

USER

Implement a Python module `reactome\_client.py` with the following:

```
Class ReactomeClient:
    __init__(self, uri, user, password): establish Neo4j driver connection
    close(self): close the driver

    get_reaction_context(self, uniprot_a: str, uniprot_b: str) -> list[dict]:
        Retrieve all human ReactionLikeEvents where both proteins co-participate.
        For each reaction return:
            { 'reaction_name': str, 'reaction_id': str, 'pathway': str,
              'role_a': str, # type(relationship) to protein A:
              'role_b': str, # type(relationship) to protein B
              'reaction_type': str } # Reaction/BlackBoxEvent/etc.

    get_protein_info(self, uniprot_id: str) -> dict:
        Return { 'display_name': str, 'gene_names': list[str], 'pathways': list[str] }
        for a given UniProt identifier.

Also write a function format_context_for_llm(context: list[dict], name_a: str,
name_b: str) -> str
that converts the raw reaction list into a clean natural language paragraph suitable
for
inclusion in an LLM prompt. Example output format:
    'In the MAPK signalling pathway, PKC (PRKCA) acts as an input and catalytic
    activator in reaction R-HSA-111933 (Phosphorylation of RAF by PKC), where RAF1
    is the output. This reaction is classified as a biochemical reaction.'
```

Include error handling for proteins not found in Reactome.
 Add a `__main__` block that tests the connection and runs
`get_reaction_context('P17252', 'P15056')`.

STEP
3

Variable Grounding Module

Foundation

Goal

Map dataset column names to UniProt IDs via LLM, validated against Reactome

Output	grounding.json: {column_name: {uniprot_id, display_name, confidence}}
Depends on	Step 2 (requires ReactomeClient.get_protein_info for validation)

Background

Variable grounding is the process of mapping raw dataset column names to canonical domain ontology concepts. For the Sachs dataset, column names like 'praf', 'pmek', and 'paks473' must be mapped to UniProt identifiers (RAF1: P15056, MAP2K1: Q02750, Akt1: P31749) before Reactome can be queried. This is a non-trivial problem in general: abbreviations, measurement conventions (the 'p' prefix denotes phosphorylated state), synonyms, and isoform ambiguities all introduce uncertainty. The grounding module formalises this step as an LLM call with structured JSON output, followed by validation against Reactome to confirm the returned identifiers exist in the database.

This is also where the system's generalisability claim is grounded: the same module, with the same prompt, will handle any new dataset in the proteomic domain without code changes. The confidence score attached to each grounding decision is propagated downstream and can attenuate constraint weights.

Implementation Tasks

- Design the grounding prompt (see below) with few-shot examples from the Sachs domain
- Call the Anthropic API in JSON mode; parse the structured response
- Validate each returned UniProt ID against Reactome using get_protein_info; mark as ungrounded if not found
- Compute a grounding confidence score (LLM self-reported + Reactome validation penalty)
- Save grounding to JSON; log any failed groundings for manual review
- Evaluate grounding accuracy against a hand-curated gold standard for the 11 Sachs variables

Agent Prompt — Variable Grounding

SYSTEM
You are a bioinformatics expert specialising in proteomics and cell signalling. Your task is to map dataset column names to their canonical UniProt protein identifiers. Rules: - Prefix 'p' typically denotes a phosphorylated measurement of the protein (e.g. praf = phospho-RAF1) - Return only human (Homo sapiens) UniProt IDs - If a name is ambiguous between isoforms, choose the most common isoform in signalling research - Report your confidence (0.0-1.0) for each mapping - If you cannot identify a protein with confidence > 0.5, return null for that entry Output ONLY a JSON object. No preamble, no markdown, no explanation outside the JSON.
USER
Dataset: {dataset_description} Domain: {domain_hint} Map each of the following column names to a UniProt protein identifier: Columns: {column_list} Return a JSON object with this exact schema: <pre>{ "<column_name>": { "uniprot_id": "<UniProt accession or null>", "canonical_name": "<human-readable protein name>",</pre>

```
    "gene_name": "<HGNC gene symbol>",  
    "confidence": <float 0.0-1.0>,  
    "reasoning": "<one sentence explaining the mapping>"  
  },  
  ...  
}
```

STEP 4	Causal Reasoning from Reaction Context	Core
-------------------	---	-------------

Goal	LLM reasons from Reactome reaction context to directed causal claims with confidence scores
Output	causal_priors.json: list of {cause, effect, confidence, constraint_type, reasoning}
Depends on	Steps 2 and 3 (requires grounding + Reactome query outputs)

Background

This is the central intellectual contribution of the system. Reactome encodes mechanistic biological knowledge in the form of typed reaction participation: a protein can be an input (consumed or modified), an output (produced or activated), a catalyst (enables the reaction without being consumed), or a regulator (activates or inhibits the reaction rate). These roles carry directional information, but translating them into causal DAG edge claims requires reasoning.

For example: if protein A is consistently an input and B is the output across multiple reactions in the same pathway, this is a strong signal that A causally precedes B. If A is a catalytic activator of a reaction that produces B, this implies $A \rightarrow B$ with high confidence. If A and B are both inputs to the same reaction, the direction is ambiguous from Reactome alone. The LLM must reason over these patterns and produce a graded confidence score, explicitly distinguishing hard constraints (confidence ≥ 0.9 , supported by multiple consistent reactions) from soft priors (confidence 0.6–0.89) and uncertain cases (< 0.6 , not used).

Implementation Tasks

- For each ordered pair (i, j) of grounded variables, call `ReactomeClient.get_reaction_context` and `format_context_for_llm`
- Call the causal reasoning prompt (see below) for each pair; parse structured JSON response
- Aggregate responses: if multiple reactions give conflicting signals, take the confidence-weighted majority
- Apply the `constraint_type` decision rule: confidence $\geq 0.9 \rightarrow$ hard constraint; 0.6–0.89 $\rightarrow$ soft prior; $< 0.6 \rightarrow$ discard
- Save full output including reasoning traces to `causal_priors.json` for auditability and ablation
- Evaluate constraint precision/recall against Sachs ground truth (separate from discovery evaluation)

Agent Prompt — Causal Reasoning

SYSTEM
You are a causal reasoning expert in molecular cell biology. You will be given structured information about how two proteins co-participate in biochemical reactions, retrieved from the Reactome pathway database. Your task is to reason about the causal direction between these two proteins and assign a confidence score. IMPORTANT REASONING RULES: - 'input $\rightarrow$ output' within the same reaction is the strongest causal signal (upstream $\rightarrow$ downstream) - 'catalyst' for a reaction where the other protein is output implies activation/causal influence

- 'regulator' of a reaction producing the other protein implies causal influence
- Both proteins as 'input' in the same reaction does NOT establish direction – return 'unknown'
- Multiple consistent reactions across pathways increase confidence
- Contradictory roles across reactions decrease confidence

Confidence scale: 0.9-1.0 = well-established mechanism; 0.7-0.89 = likely based on pathway context;

0.5-0.69 = plausible but uncertain; < 0.5 = return direction as

'unknown'

Output ONLY valid JSON. No preamble.

USER

Protein A: {protein_a_name} ({uniprot_a})

Protein B: {protein_b_name} ({uniprot_b})

Reactome reaction context:

{formatted_context}

Based on this evidence, determine the most likely causal direction between these two proteins.

Return a JSON object with this exact schema:

```
{
  "cause": "<protein_a_name or protein_b_name or 'unknown'>",
  "effect": "<protein_a_name or protein_b_name or 'unknown'>",
  "confidence": <float 0.0-1.0>,
  "constraint_type": "<'hard_required' | 'soft_prior' | 'hard_forbidden_reverse' | 'unknown'>",
  "supporting_reactions": [<"<reaction name 1>", ...>],
  "contradicting_reactions": [<"<reaction name or empty list>">],
  "reasoning": "<2-3 sentence explanation of the causal inference>"
}
```

**STEP
5**

Constraint Integration and Constrained Discovery

Core

Goal

Translate causal priors into causal-learn BackgroundKnowledge; run constrained PC/GES/LiNGAM

Output

discovery_results.json with metrics for all (algorithm × condition) combinations

Depends on

Steps 1, 3, 4 (requires baseline harness + grounding + causal priors)

Background

The causal-learn library provides a BackgroundKnowledge API that accepts required and forbidden edges specified by variable index. Hard constraints (confidence ≥ 0.9) are passed directly as required or forbidden edges. Soft priors (confidence 0.6–0.89) are handled differently depending on the algorithm: for PC and GES they are treated as additional required edges with a lower-confidence flag; for LiNGAM they are encoded as a prior_knowledge matrix (1 = required, -1 = forbidden, 0 = unknown). The system must handle the translation cleanly and log exactly which constraints were applied for each run.

Implementation Tasks

- Implement ConstraintBuilder: accepts causal_priors.json + column names → BackgroundKnowledge + LiNGAM prior matrix
- Implement a threshold parameter to sweep the confidence cutoff (0.6, 0.7, 0.8, 0.9)
- Run each algorithm under all ablation conditions (see Step 6); save all graphs and metrics
- Implement graph serialisation: save predicted DAGs as adjacency matrices and edge lists for reproducibility

Agent Prompt — Constraint Integration Module

SYSTEM

You are a Python engineer implementing a causal discovery pipeline using the causal-learn library.
You have access to causal_priors.json (list of {cause, effect, confidence, constraint_type}) and a column name list for the dataset.
Your goal is to translate the priors into the formats expected by causal-learn's three algorithms.

USER

Implement a Python module `constraint\_builder.py` with the following:

```
Class ConstraintBuilder:
    __init__(self, causal_priors: list[dict], columns: list[str], confidence_threshold: float = 0.7)
```

```
    build_background_knowledge(self) -> BackgroundKnowledge:
```

```
        From causal_learn.utils.PCUtils.BackgroundKnowledge import BackgroundKnowledge
```

```
        For each prior where confidence >= self.confidence_threshold:
```

```
            - constraint_type == 'hard_required': add_required_by_node(i, j)
```

```
            - constraint_type == 'hard_forbidden_reverse': add_forbidden_by_node(j, i)
```

```
            - constraint_type == 'soft_prior' and confidence >= threshold:
```

```
add_required_by_node(i, j)
```

```
    Return the BackgroundKnowledge object.
```

```
    build_lingam_prior_matrix(self) -> np.ndarray:
```

```
    Return an (n x n) numpy array where:
```

```
        prior[i][j] = 1 if edge i->j is required (confidence >= threshold)
```

```
        prior[i][j] = -1 if edge i->j is forbidden
```

```
        prior[i][j] = 0 otherwise
```

```
    This is the prior_knowledge parameter for DirectLiNGAM.
```

```
    get_constraint_summary(self) -> dict:
```

```
        Return { 'hard_required': int, 'hard_forbidden': int, 'soft_prior': int,
'discarded': int }
```

```
Include a run_all_algorithms(data, bk, lingam_prior) -> dict function that runs PC, GES, and DirectLiNGAM and returns a dict of {algorithm: networkx.DiGraph}.
```

STEP 6	Ablation Study and Evaluation	Evaluation
-------------------	--------------------------------------	-------------------

Goal	Quantify the contribution of each pipeline component; produce publication-ready results
Output	ablation_results.json; results tables and figures for the paper
Depends on	Steps 1–5 (all modules complete)

Background

The ablation study is the primary experimental contribution of the paper. It directly answers the question: how much does structured, ontology-derived domain knowledge improve causal discovery accuracy? The design follows the template established in Jarosławski et al. (RAI 2025), extended with the additional knowledge conditions introduced by this system. Each condition is run 10 times with fixed seeds to report mean $\pm$ std.

Ablation Conditions

#	Condition	Description
C0	No Knowledge (Baseline)	PC / GES / LiNGAM with no domain knowledge constraints. Identical to Jarosławski et al. (2025) setup on LUCAS; here applied to Sachs.
C1	Hard Constraints Only	Only constraints with confidence ≥ 0.9 are used as required/forbidden edges. Tests whether high-certainty ontology edges alone improve results.
C2	Hard + Soft Priors ($\tau=0.7$)	All constraints with confidence ≥ 0.7 are used. The full knowledge extraction output at a moderate confidence threshold.
C3	Hard + Soft Priors ($\tau=0.6$)	All constraints with confidence ≥ 0.6 are used. Tests sensitivity to threshold: lower threshold = more constraints = higher risk of incorrect priors.
C4	Oracle Upper Bound	True graph edges used as constraints. Establishes the maximum possible improvement from perfect domain knowledge. Not achievable in practice but necessary for interpretation.

Constraint Quality Evaluation (Independent of Discovery)

Before evaluating discovery performance, evaluate the quality of the extracted constraints in isolation. This is a novel secondary contribution: measuring how accurately an LLM can infer causal direction from ontological reaction context.

- Precision of extracted edges: fraction of predicted causal edges that match the ground truth
- Recall of extracted edges: fraction of ground-truth edges that are covered by extracted constraints
- Hallucination rate: fraction of extracted edges that directly contradict the ground truth (forbidden edges in the wrong direction)
- Coverage rate: fraction of Sachs variable pairs for which Reactome returned non-empty reaction context

Agent Prompt — Ablation Runner and Results Reporter

SYSTEM
You are a research engineer running a systematic ablation study for a causal discovery pipeline. You have modules: <code>baseline.py</code>, <code>reactome_client.py</code>, <code>grounding.py</code>, <code>causal_reasoning.py</code>, <code>constraint_builder.py</code>. All have been implemented and tested. Your task is to orchestrate the full ablation, collect results, and produce analysis outputs.
USER
Implement <code>`run_ablation.py`</code> that executes the full ablation study: Conditions to run (for all 3 algorithms × 10 seeds each): - C0: no background knowledge - C1: constraints with confidence ≥ 0.9 - C2: constraints with confidence ≥ 0.7 - C3: constraints with confidence ≥ 0.6 - C4: oracle (ground-truth edges as required constraints) For each (condition × algorithm × seed) run: - Build BackgroundKnowledge / LiNGAM prior matrix for the condition - Run the algorithm on Sachs data - Evaluate: SHD, AUPR, Precision, Recall, F1 - Store result with full metadata Save all results to <code>ablation_results.json</code>. Also implement <code>`report_results.py`</code> that: - Loads <code>ablation_results.json</code> - Prints a LaTeX-formatted table (mean ± std) for each metric × condition × algorithm - Prints the constraint quality evaluation table (precision, recall, hallucination rate) - Identifies which condition achieves best F1 for each algorithm and by how much compared to the no-knowledge baseline (C0)

STEP 7	Analysis, Paper Writing, and Reproducibility	Evaluation
Goal	Interpret results scientifically; produce paper-ready figures and a reproducible code repository	
Output	Paper draft sections (Introduction, Method, Experiments, Results, Discussion); GitHub repository with Docker setup	
Depends on	Step 6 (requires ablation results)	

Key Research Questions to Address in the Paper

- How much does ontology-derived domain knowledge improve causal discovery accuracy over no-knowledge baselines, and which algorithm benefits most?

- What fraction of the Sachs ground-truth edges is recoverable from Reactome reaction context alone, and what is the hallucination rate?
- How sensitive is performance to the confidence threshold τ ? Is there a sweet spot, or does more knowledge always help?
- How does the system's performance compare to CausalCopilot's LLM-pruned output (where available from their paper)?

Reproducibility Checklist

- docker-compose.yml to launch Reactome Neo4j in one command
- requirements.txt with pinned versions for all Python dependencies
- Fixed random seeds in all stochastic components
- Full grounding.json and causal_priors.json committed to repository for inspection
- All ablation results and figures regenerable from a single script: python run_ablation.py && python report_results.py

Agent Prompt — Scientific Analysis and Discussion

SYSTEM

You are a PhD-level scientific writing collaborator specialising in causal discovery and explainable AI.
 You have access to ablation_results.json, constraint_quality.json, and the Sachs ground truth graph.
 Write in formal academic English suitable for a top ML conference (UAI, ECAI, or AISTATS).
 Be precise, concise, and avoid claims not supported by the results.

USER

Given the following ablation results: {ablation_results_summary}
 And constraint quality metrics: {constraint_quality_summary}

Write the following sections of the paper:

1. RESULTS section (400-500 words):
 - Report baseline performance on Sachs for PC, GES, LiNGAM (condition C0)
 - Report the improvement from each knowledge condition (C1-C3) per algorithm
 - Report constraint quality metrics: precision, recall, hallucination rate
 - Include the LaTeX table from report_results.py
 - Note any surprising or counter-intuitive findings
2. DISCUSSION section (300-400 words):
 - Explain WHY certain algorithms benefited more from domain knowledge (based on their assumptions)
 - Discuss the hallucination rate: what does it mean for practical use?
 - Discuss the confidence threshold sensitivity: is $\tau=0.7$ a robust choice?
 - Position results relative to CausalCopilot and ALCM from the related work
 - State limitations clearly: single dataset, single biological domain, English-only ontology
 - Suggest two concrete directions for future work

Use \cite{} placeholders for all references.

4. Step Dependencies and Timeline

Step	Name	Est. Effort	Blocking Steps
1	Env Setup and Sachs Baseline	0.5–1 day	None
2	Reactome Neo4j Setup and Schema	0.5–1 day	Step 1 (need Sachs variable names)
3	Variable Grounding Module	1 day	Step 2 (Reactome validation)
4	Causal Reasoning from Reactome	2 days	Steps 2 + 3 (requires both)
5	Constraint Integration and Discovery	1 day	Steps 1 + 3 + 4
6	Ablation Study and Evaluation	1–2 days	Steps 1–5 all complete
7	Analysis, Writing, Reproducibility	3–5 days	Step 6 (requires results)

Steps 1 and 2 can be parallelised (no shared dependency). Step 3 depends on Step 2 but not Step 1. The critical path is: Step 2 → Step 3 → Step 4 → Step 5 → Step 6 → Step 7.

5. Key References

- Sepiolo, D. & Ligeza, A. Towards Model-Driven Explainable Artificial Intelligence: Function Identification with Grammatical Evolution. *Applied Sciences*, 14(13):5950, 2024.
- Jaroslawski, M., Sepiolo, D. & Ligeza, A. From Data to Decisions: Comparing Causal Discovery Methods on a Benchmark Dataset. *RAI* 2025.
- Sepiolo, D., Jaroslawski, M. & Ligeza, A. Grammar Refinement in Grammatical Evolution Using Large Language Models. *RAI* 2025.
- Wang et al. CausalCopilot: An Agentic System for Automated Causal Analysis. *arXiv:2504.13263*, 2025.
- Milacic et al. The Reactome Pathway Knowledgebase 2024. *Nucleic Acids Research*, 2023.
- Sachs et al. Causal Protein-Signaling Networks Derived from Multiparameter Single-Cell Data. *Science*, 308(5721):523–529, 2005.
- Zheng et al. Causal-learn: Causal Discovery in Python. *JMLR*, 25(60):1–8, 2024.
- Shimizu et al. A Linear Non-Gaussian Acyclic Model for Causal Discovery. *JMLR*, 7:2003–2030, 2006.